

Módulo 4 – Capítulo 06 – Sección 01

Observabilidad en Sistemas de IA

Hay una diferencia fundamental entre un sistema que funciona y un sistema del que se sabe que está funcionando. Un sistema de IA puede estar produciendo respuestas incorrectas desde hace días sin que ningún componente de infraestructura reporte un error: la latencia es normal, el CPU está en rangos aceptables, las llamadas a la API del LLM retornan HTTP 200. El problema no es un fallo técnico — es una degradación en la calidad del output que ningún monitor de infraestructura convencional puede detectar porque la calidad de lenguaje natural no es una métrica de infraestructura. Este es el problema central que la observabilidad en sistemas de IA debe resolver.

La observabilidad, en el sentido clásico de sistemas distribuidos, es la capacidad de inferir el estado interno de un sistema a partir de sus outputs externos: métricas, logs y trazas. En sistemas de software determinístico, ese estado interno es preciso: el servidor web está en estado "saludable" o "fallido", la base de datos tiene N conexiones activas, el tiempo de respuesta del endpoint es de X milisegundos. En sistemas de IA, hay una tercera categoría de estado que esos mecanismos clásicos no capturan: el estado de la calidad del comportamiento del modelo. Un LLM puede estar respondiendo en 300 milisegundos con un HTTP 200 y aún así estar produciendo respuestas que no responden a la pregunta del usuario, que contienen información incorrecta, o que han cambiado en tono y estilo respecto a la semana anterior. La observabilidad de sistemas de IA debe cubrir todas esas dimensiones.

La arquitectura de observabilidad de un sistema de IA productivo se organiza en tres capas complementarias:

Trazas distribuidas: registran el flujo completo de cada interacción del usuario a través de todos los componentes del sistema. En un sistema RAG, una traza completa incluye la consulta del usuario, el embedding generado, los resultados de la búsqueda vectorial, el output del reranker, el prompt final enviado al LLM, y la respuesta generada. En un sistema de agentes, la traza incluye cada iteración del bucle de razonamiento: qué acción decidió el modelo, qué herramienta fue invocada, qué observación fue recibida. Plataformas como

LangSmith (profundamente integrado con LangChain), Langfuse (open source, soporte para cualquier framework), y Phoenix de Arize (enfocado en evaluación) proporcionan esta capacidad de trazado específicamente para sistemas de IA. Para la infraestructura general, OpenTelemetry con exporters a Jaeger o Grafana Tempo es el estándar.

Métricas: datos numéricos que representan el estado del sistema en dimensiones técnicas y funcionales. Las métricas de infraestructura — latencia, disponibilidad, tasa de error, tokens consumidos — son el piso mínimo. Las métricas de calidad del sistema de IA — precisión de recuperación, faithfulness de las respuestas, frecuencia de escaladas a humano — son las que revelan la salud real del sistema. Datadog LLM Observability y similares ofrecen dashboards especializados para estas métricas.

Logs estructurados: registros en formato legible por máquina (JSON) que capturan eventos discretos del sistema con su contexto completo. A diferencia de los logs de texto libre, los logs estructurados pueden consultarse, filtrarse y agregarse con herramientas estándar (Elasticsearch, Loki, CloudWatch). En sistemas de IA, los logs deben incluir los identificadores de correlación que conectan todos los registros de una misma interacción del usuario a través de los distintos componentes.

El principio rector de la observabilidad en sistemas de IA es la auditabilidad del comportamiento: para cualquier output del sistema, debe ser posible reconstruir exactamente qué inputs, qué contexto, qué razonamiento y qué pasos de ejecución lo produjeron. Sin esa auditabilidad, los problemas de calidad solo pueden diagnosticarse a través del síntoma (la respuesta incorrecta) sin poder identificar la causa raíz (en qué etapa del pipeline ocurrió el error). Las secciones siguientes desarrollan cada capa de la observabilidad con herramientas específicas, criterios de diseño y los patrones de fallos que cada capa permite detectar.

Módulo 4 – Capítulo 06 – Sección 02

Métricas Técnicas

Las métricas técnicas de un sistema de IA son la capa de observabilidad que responde a la pregunta: ¿el sistema está operando dentro de sus parámetros de diseño? No dicen si el sistema está produciendo buenas respuestas — eso es dominio de las métricas de calidad — sino si los componentes de infraestructura están funcionando correctamente. Son la base indispensable sobre la que se construye cualquier análisis de problemas más sofisticado.

Latencia por etapa es la métrica de mayor utilidad operativa en sistemas de IA con pipelines complejos. La latencia total percibida por el usuario es la suma de múltiples contribuciones: tiempo de embedding de la consulta, tiempo de búsqueda en la base vectorial, tiempo de reranking, tiempo de construcción del prompt, tiempo de generación del LLM (time-to-first-token y tiempo de generación completa), y tiempo de procesamiento post-generación. Instrumentar cada etapa por separado permite identificar dónde ocurre la degradación cuando la latencia total aumenta. Una latencia elevada de búsqueda vectorial indica necesidad de optimización del índice o escalado de la base vectorial. Una latencia elevada de generación puede indicar un modelo congestionado, un contexto demasiado extenso, o necesidad de optimización de la capa de inferencia. La latencia debe expresarse en percentiles: P50 (mediana), P95 y P99 revelan el comportamiento del sistema ante la mayoría de los usuarios y ante los casos extremos.

Consumo de tokens es la métrica de costo directo en sistemas de IA basados en APIs de LLMs comerciales. Debe medirse en tres dimensiones: tokens de entrada por solicitud (contexto + consulta), tokens de salida por solicitud (respuesta generada), y costo monetario derivado. El tracking de tokens por solicitud, por usuario, por tipo de consulta y por período temporal permite identificar los patrones de uso más costosos y tomar decisiones informadas sobre optimización: reducir el tamaño del contexto, usar un modelo más económico para categorías de consultas simples, o implementar caché semántico para consultas frecuentes. Herramientas como LangSmith y Langfuse calculan automáticamente el costo de tokens usando las tarifas actuales de cada proveedor.

Tasa de error por tipo debe distinguir entre categorías de error con causas y soluciones distintas. Los errores de timeout (la llamada al LLM supera el tiempo máximo configurado) sugieren congestión del proveedor o contextos demasiado extensos. Los errores de rate limit (el sistema supera la cuota de solicitudes por minuto del proveedor) sugieren necesidad de queue con backoff exponencial o distribución entre múltiples claves de API. Los errores de formato de respuesta (el modelo no sigue el schema JSON solicitado) sugieren problemas en el design del prompt de salida. Los errores de herramienta en sistemas de agentes (una tool invocation falla) requieren logging del input exacto enviado a la herramienta para diagnóstico.

Disponibilidad del servicio mide el porcentaje de solicitudes que completan exitosamente en el tiempo máximo configurado. En sistemas de IA con dependencias de APIs externas (OpenAI, Anthropic, Cohere), la disponibilidad propia es altamente dependiente de la disponibilidad de los proveedores. Un circuit breaker bien configurado — que detecta cuando el proveedor principal tiene alta tasa de error y redirige automáticamente a un proveedor secundario — es un mecanismo de alta disponibilidad específico de sistemas de IA que no tiene equivalente directo en sistemas tradicionales.

Métricas de la base vectorial incluyen el tiempo de indexación (cuánto tarda en procesar nuevos documentos), el tiempo de búsqueda por percentil (fundamental para la latencia del pipeline RAG), el uso de memoria del índice, y la tasa de actualización (documentos procesados por hora). Weaviate, Qdrant y Pinecone exponen estas métricas a través de sus interfaces de monitoreo nativas, que pueden integrarse con Prometheus/Grafana para dashboards unificados.

El stack de observabilidad técnica recomendado para sistemas de IA en producción combina componentes especializados con componentes estándar: **Langfuse** o **LangSmith** para trazas y métricas específicas de LLM (tokens, latencia por llamada, costos, evals integradas), **OpenTelemetry** con exporters a **Grafana Tempo** o **Jaeger** para trazas distribuidas de la infraestructura general, **Prometheus** + **Grafana** para métricas de infraestructura de la base vectorial y servicios de soporte, y **Datadog LLM Observability** o equivalente para dashboards ejecutivos.

Nota del Arquitecto: El error más frecuente en la instrumentación de sistemas de IA es medir solo la latencia de la llamada al LLM y asumir que eso es la latencia total. He visto casos donde la latencia del LLM es de 1.2 segundos pero el usuario experimenta 4.8 segundos, porque nadie había medido que el reranker añadía 2.5

segundos y la construcción del prompt añadía 1.1 segundos en casos con muchos chunks. Siempre instrumente cada etapa del pipeline por separado desde el día uno.

Las métricas técnicas son el idioma en que el sistema habla sobre su propio estado. Aprenderlo completamente — incluyendo los acentos y los silencios — es la primera habilidad de un equipo de operaciones de IA maduro. La siguiente sección explora las métricas que traducen ese estado técnico al idioma del negocio.

Módulo 4 – Capítulo 06 – Sección 03

Métricas de Negocio

Las métricas técnicas responden preguntas sobre la ingeniería del sistema. Las métricas de negocio responden preguntas sobre su valor: ¿el sistema está ayudando a los usuarios a alcanzar sus objetivos? ¿la calidad percibida de las respuestas está mejorando o deteriorándose con el tiempo? ¿el sistema está generando el impacto económico que justificó su construcción? En organizaciones maduras, la salud de un sistema de IA se evalúa por ambas categorías de métricas en un único dashboard ejecutivo, y las decisiones sobre inversión y mejora del sistema se toman sobre la base de ambas.

Precisión percibida es la métrica de calidad más directa: mide si el usuario considera que la respuesta del sistema respondió correctamente a su pregunta. El mecanismo más simple es el feedback explícito: thumbs up / thumbs down al final de cada respuesta, o una escala de calificación de 1 a 5. El problema del feedback explícito es el bajo índice de participación — la mayoría de los usuarios no califica las respuestas, especialmente las correctas. Una métrica proxy más fiable es la tasa de reformulación: cuando el usuario envía inmediatamente una segunda consulta que es semánticamente equivalente a la primera, es una señal de que la respuesta no fue satisfactoria. Esta señal puede detectarse automáticamente comparando los embeddings de consultas consecutivas del mismo usuario.

Tasa de escalada mide con qué frecuencia el sistema deriva al usuario a un humano (en sistemas de soporte) o declara que no puede responder a la consulta. Una tasa de escalada baja puede indicar que el sistema responde bien, o que está respondiendo de forma incorrecta sin detectar su propia limitación. Una tasa de escalada alta indica que hay consultas frecuentes para las que el sistema no tiene conocimiento o capacidad suficiente — información valiosa para planificar mejoras en la base de conocimiento o en las capacidades del agente.

Tasa de finalización de tarea es la métrica más relevante para sistemas de agentes: ¿qué porcentaje de las tareas iniciadas por el usuario son completadas exitosamente por el agente? Esta métrica debe distinguir entre tareas completadas sin intervención humana, tareas

completadas con intervención humana, y tareas abandonadas (el usuario cerró la sesión sin recibir resultado). La tendencia en el tiempo de la tasa de finalización es más informativa que el valor puntual: si la tasa de finalización sin intervención humana crece mes a mes, el agente está aprendiendo implícitamente del feedback y las mejoras del sistema.

Valor generado por el sistema es la métrica más difícil de medir pero la más importante para los tomadores de decisión. Dependiendo del caso de uso, puede expresarse como: horas-hombre ahorradas (un agente de soporte que resuelve X tickets que antes requería Y horas de trabajo humano), incremento en conversión (un chatbot de ventas que incrementa la tasa de conversión en Z%), o reducción de costos operativos (el costo de atender una consulta con el sistema de IA versus el costo de atenderla con un humano). Establecer esta métrica requiere instrumentar no solo el sistema de IA sino también los procesos de negocio que rodean al sistema — lo que es más trabajo pero produce el argumento más poderoso para la inversión continua.

Drift de calidad es la detección de cambios gradualmente en la calidad del comportamiento del sistema a lo largo del tiempo. El drift puede ocurrir por múltiples causas: el vocabulario del dominio evoluciona y los documentos antiguos de la base de conocimiento no lo reflejan, el modelo de embeddings del proveedor cambia silenciosamente en una actualización, el perfil de las consultas de los usuarios cambia y el sistema no está preparado para los nuevos patrones. El drift se detecta midiendo periódicamente las métricas de calidad (RAGAS u otras) sobre un conjunto de evaluación fijo y comparando con la línea base. Un drift detectado tempranamente puede corregirse con ajustes en el pipeline; un drift detectado tarde produce quejas de usuarios y pérdida de confianza.

Satisfacción del usuario puede medirse a través de surveys periódicos cortos, NPS (Net Promoter Score) adaptado al producto, o análisis de sentimiento sobre los comentarios de feedback libre que los usuarios proporcionan. Estas métricas tienen baja frecuencia (no se miden por sesión sino por período) pero alta señal: capturan la percepción global del usuario sobre el sistema, incluyendo aspectos que las métricas automáticas no pueden medir, como la confianza que genera el sistema o la satisfacción con su personalidad y estilo de comunicación.

Las métricas de negocio no son responsabilidad exclusiva del equipo de datos o de IA: requieren colaboración con los equipos de producto, operaciones y finanzas para definir qué se mide, cómo se mide y cuáles son los umbrales de alarma. Un sistema de IA sin métricas de negocio es técnicamente observable pero organizacionalmente ciego.

Módulo 4 – Capítulo 06 – Sección 04

Alertas y Respuesta

Recopilar métricas sin definir alertas es observabilidad sin consecuencias: los datos existen, pero el equipo no es notificado cuando algo sale mal hasta que el problema ya ha afectado a los usuarios. Las alertas son el mecanismo que convierte la observabilidad pasiva en gestión proactiva. Su diseño requiere tanto rigor técnico como disciplina organizacional: alertas demasiado sensibles producen fatiga de alertas (el equipo las ignora porque la mayoría son falsos positivos); alertas con umbrales demasiado permisivos detectan los problemas demasiado tarde.

El diseño de alertas para sistemas de IA debe abordar las siguientes categorías:

Alertas de infraestructura (umbrales absolutos): son las más simples y las más maduras del arsenal de alertas. La latencia P95 supera los 5 segundos, la tasa de error supera el 5%, la disponibilidad del servicio cae por debajo del 99%. Estas alertas se configuran con herramientas estándar (Grafana Alerting, Datadog Monitors, CloudWatch Alarms) y su respuesta es operativa: reiniciar un servicio, escalar recursos, activar el proveedor de LLM de respaldo.

Alertas de costo (umbrales de gasto): son específicas de sistemas de IA. Un pico anómalo en el consumo de tokens puede indicar un agente en bucle, una consulta maliciosa que genera contextos muy extensos, o un error en el pipeline que replica solicitudes. Alertas de gasto diario o por hora con umbrales definidos sobre la línea base histórica permiten detectar estas anomalías antes de que produzcan facturas inesperadas.

Alertas de calidad (umbrales estadísticos): son las más complejas y las más valiosas. Cuando las métricas RAGAS ejecutadas periódicamente muestran que el context recall ha caído por debajo de un umbral histórico, o cuando la tasa de reformulación de consultas supera el doble de su valor de referencia, una alerta debe notificar al equipo. Estas alertas requieren la ejecución periódica (diaria o semanal) del pipeline de evaluación sobre el conjunto de test fijo.

Alertas de drift de modelo: cuando el proveedor de LLM actualiza el modelo subyacente (algo que ocurre sin aviso en versiones no fijadas como `gpt-4o` vs. `gpt-4o-2024-11-20`), el comportamiento del sistema puede cambiar de forma que las métricas de infraestructura no detectan. Comparar el output del modelo ante un conjunto de prompts de referencia con sus outputs históricos esperados permite detectar cambios de comportamiento silenciosos.

Los **Service Level Objectives (SLO)** para sistemas de IA merecen una discusión específica porque son fundamentalmente distintos a los SLOs de sistemas determinísticos. Un sistema de software tradicional puede comprometerse a que el 99.9% de las solicitudes se completen en menos de 200ms, porque el output es determinístico. Un sistema de IA no puede comprometerse a que el 99.9% de las respuestas sean correctas — la calidad del output es probabilística. Los SLOs adecuados para sistemas de IA son los que miden calidad estadística a lo largo del tiempo: "el promedio de faithfulness medido semanalmente sobre el conjunto de evaluación no será inferior a 0.85", o "la tasa de escalada no superará el 15% en ningún período de 24 horas". Estos SLOs requieren el establecimiento previo de líneas base realistas sobre el comportamiento real del sistema.

El **runbook de respuesta a incidentes** de un sistema de IA debe incluir procedimientos específicos que van más allá del runbook estándar de operaciones:

- Cómo verificar si un cambio de calidad es causado por el LLM (problema del proveedor), el retriever (problema del índice vectorial), la base de conocimiento (documentos desactualizados) o el prompt (cambio accidental en el template).
- Cómo activar el modo de respaldo: un modelo alternativo, una versión fija del sistema anterior, o degradación controlada (responder solo las consultas simples y escalar las complejas).
- Cómo comunicar la degradación a los usuarios cuando el sistema está operando con calidad reducida.
- Los criterios de rollback: bajo qué condiciones el sistema revierte automáticamente a la versión anterior del pipeline.

Nota del Arquitecto: La fatiga de alertas es el mayor enemigo de la observabilidad. En sistemas donde la latencia tiene alta varianza natural, una alerta configurada con un umbral demasiado bajo dispara varias veces al día sin indicar ningún problema real. El equipo aprende a ignorarla — y cuando la alerta correcta dispara, también la ignoran. Diseñe las alertas con la misma disciplina con que

diseña el sistema: definición clara de qué mide, umbral justificado por datos históricos, y propietario responsable de la respuesta.

Un sistema con alertas bien diseñadas se comporta como un sistema autovigilado: el equipo sabe cuándo el sistema está saludable no porque estén mirando el dashboard constantemente, sino porque cuando algo sale de los parámetros esperados, el sistema se lo dice.

Módulo 4 – Capítulo 06 – Sección 05

Auditoría y Cumplimiento Operativo

La auditoría en sistemas de IA tiene dos motivaciones distintas que con frecuencia se confunden. La primera es operativa: la capacidad de reconstruir exactamente qué ocurrió en el sistema para diagnosticar un problema, mejorar la calidad o explicar una decisión del sistema. La segunda es regulatoria: la demostración a auditores externos de que el sistema opera dentro de las normas aplicables. Esta sección trata la auditoría operativa — el registro técnico de interacciones que permite la gestión y mejora del sistema. El cumplimiento regulatorio organizacional se desarrolla en el Capítulo 09.

El registro de interacciones es el cimiento de la auditoría operativa. Cada consulta de usuario que el sistema procesa debe generar un registro permanente que incluya: el identificador único de la interacción, el timestamp con precisión de milisegundos, el identificador del usuario (anonimizado o no, según la política de privacidad del sistema), el texto completo de la consulta, los identificadores y puntuaciones de los chunks recuperados, el prompt completo enviado al LLM (incluyendo el contexto), la respuesta generada, el modelo utilizado, la versión del pipeline, y el costo total de tokens de la interacción. Este registro permite, ante cualquier incidente ("el sistema respondió X, que era incorrecto"), reconstruir la cadena causal completa.

La trazabilidad de fuentes es el aspecto de la auditoría operativa más relevante para gestionar la calidad del conocimiento. Si el sistema produce una respuesta incorrecta y el registro muestra qué documentos fueron recuperados para fundamentarla, el equipo puede identificar si el error viene de un documento desactualizado, un documento con información incorrecta, o un fallo del retriever que recuperó el documento equivocado. Esta trazabilidad también permite identificar qué documentos de la base de conocimiento son más frecuentemente citados y cuáles nunca son recuperados — información valiosa para la curación de la base de conocimiento.

El análisis de patrones de consultas es la aplicación de la auditoría para la mejora continua. Las consultas archivadas — anonimizadas para proteger la privacidad del usuario — permiten

identificar: los temas sobre los que los usuarios consultan con más frecuencia (para priorizar la expansión de la base de conocimiento), los tipos de consultas que producen respuestas de baja calidad con mayor frecuencia (para identificar lagunas del sistema), y los cambios en el perfil de uso a lo largo del tiempo (para anticipar nuevas capacidades necesarias). Este análisis es el input más valioso para el roadmap de mejora del sistema.

El nivel de retención de los logs de interacción debe balancear dos fuerzas opuestas: el valor operativo de conservar el historial completo (para análisis de tendencias y mejora del sistema) y el costo de almacenamiento y privacidad de conservar conversaciones de usuarios durante períodos extensos. La política típica en sistemas empresariales es: retención completa durante 90 días para diagnóstico operativo, retención de estadísticas agregadas y anonimizadas indefinidamente, y eliminación de datos identificables de usuario según las políticas de retención de datos de la organización.

El control de acceso a los logs de auditoría es tan importante como el control de acceso al sistema principal. Los logs de interacción pueden contener información sensible de los usuarios (las preguntas que hacen, los documentos que consultan, la información que revelan en sus consultas). Solo el personal con necesidad legítima y autorización explícita debe poder acceder a logs que contienen datos identificables. El acceso a los logs debe ser registrado en un log de auditoría de segundo nivel — el "log del log" — que permite verificar quién accedió a qué información de usuario.

La generación de reportes de uso periódicos — resúmenes del volumen de consultas, tipos de uso, calidad medida, costo acumulado — convierte los datos de auditoría en información de gestión. Estos reportes son la base de las conversaciones entre el equipo técnico y los stakeholders de negocio: demuestran el valor del sistema, identifican áreas de mejora y justifican inversiones en capacidad y calidad.

El cumplimiento operativo no es burocracia: es la capacidad de operar el sistema con confianza. Un equipo que sabe exactamente qué hace su sistema en cada interacción, que puede responder cualquier pregunta sobre su comportamiento con datos y no con suposiciones, y que tiene la trazabilidad completa de cada output, es un equipo que puede operar con autonomía responsable.

Módulo 4 – Capítulo 06 – Sección 06

Resumen

Este capítulo desarrolló la observabilidad como la disciplina que convierte un sistema de IA funcional en un sistema de IA gestionable. La diferencia no es trivial: un sistema funcional puede operar correctamente durante semanas y degradarse silenciosamente hasta el punto de fallar de manera consistente sin que ningún dashboard de infraestructura convencional lo detecte. Un sistema observable, en cambio, revela su estado en tiempo real a través de métricas, trazas y logs que permiten al equipo detectar problemas antes de que afecten a los usuarios.

La observabilidad técnica — latencia por etapa, consumo de tokens, tasa de error por tipo, disponibilidad — es el piso mínimo. El stack específico de LLMs (LangSmith, Langfuse, Phoenix de Arize) más la infraestructura general (OpenTelemetry, Prometheus, Grafana) cubre esta capa. Pero la observabilidad técnica no es suficiente: un sistema puede tener latencia de 300ms y tasa de error del 0.1% y aún así producir respuestas incorrectas el 30% del tiempo porque el retriever está recuperando chunks irrelevantes. Por eso la capa de métricas de calidad — context recall, faithfulness, tasa de reformulación, drift de comportamiento — es igualmente indispensable.

Las métricas de negocio — precisión percibida, tasa de escalada, tasa de finalización, valor generado, satisfacción del usuario — traducen el estado técnico del sistema al idioma de los tomadores de decisión. Son las métricas que responden a la pregunta que los stakeholders de negocio realmente hacen: ¿vale la pena continuar invirtiendo en este sistema? Sin métricas de negocio, la respuesta siempre es una estimación subjetiva.

El diseño de alertas convierte la observabilidad pasiva en gestión proactiva. Las alertas deben estar calibradas para detectar problemas reales sin generar fatiga: umbrales basados en datos históricos, propietarios responsables de la respuesta y runbooks que incluyen procedimientos específicos para los tipos de incidentes más frecuentes en sistemas de IA. Los SLOs deben adaptarse a la naturaleza probabilística del output — medir calidad estadística a lo largo del tiempo, no garantías absolutas por interacción.

La auditoría operativa — el registro completo de cada interacción con su trazabilidad de fuentes y cadena causal — es lo que permite el ciclo de mejora continua del sistema: identificar lagunas en la base de conocimiento, detectar patrones de consultas sin respuesta adecuada, y responder con datos ante cualquier pregunta sobre el comportamiento del sistema.

El principio que une todas las capas de la observabilidad es simple pero exigente: si una solución no puede observarse ni medirse, tampoco puede mejorarse de forma sistemática. El sistema de IA que no mide su calidad está evolucionando a ciegas.

El Capítulo 07 aborda la segunda disciplina operativa crítica: la seguridad. A diferencia de la observabilidad, que responde a la pregunta "¿está funcionando correctamente?", la seguridad responde a la pregunta "¿puede ser comprometido?". Las respuestas a esas dos preguntas son igualmente necesarias para operar un sistema de IA en producción con confianza.

"Si una solución no puede observarse ni medirse, tampoco puede mejorarse de forma sistemática."

— Principio de observabilidad en ingeniería de sistemas